

项目介绍

编程语言：Python

软件需求：Anaconda + cuda

项目地址： <https://github.com/ryougishiki02>

DeepEM

```
└─ image
  │ └─ element_sort
  │   └─ • element.py
  │ └─ strain
  │   │ └─ • pkg.py
  │   └─ • strain.py
  │ └─ test
  │   └─ • remove_background.py
  │   └─ • remove_denoise.py
  │   └─ • pkg.py
└─ video
  │ └─ • crop.py
  │ └─ • refine.py
└─ model
  │ └─ unet
  │   │ └─ unet3
  │   │   │ └─ • denoise
  │   │   └─ • segment
  │   └─ unet5
  │       │ └─ • denoise
  │       └─ • segment
  │       └─ • pkg.py
└─ • requirements.txt
```

图像去背底、降噪、元素分析、应变分析

视频裁剪、抽帧（后续会增加降噪和漂移矫正）

神经网络以及参数

具体环境要求

目录

1、STEM图像处理

1.1 去背底

1.2 降噪

1.3 应变分析

1.4 元素分析

STEM图像去背底

```
remove_background.py ×  
1 from image.pkg import Backgroundremover, load_img  
2  
3 image = load_img()  
4  
5 # remove background  
6 # Define the scaling parameters and remove the background from the image  
7 remover = Backgroundremover(image, max_scale=2048, min_scale=32)  
8 image = remover.remove()  
9 remover.visualize() # 可视化去背景后的图像  
10 remover.save() # 保存去背景后的图像
```

适用范围：STEM、TEM图像
(任意尺寸)

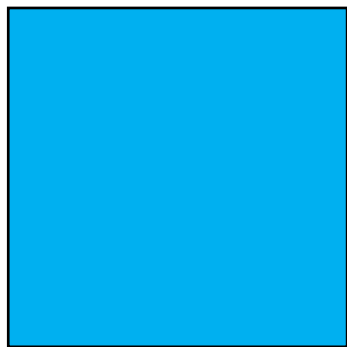
调整参数：最大及最小图像尺寸

使用方法：右键—运行

运行 'remove_background (1...)'(U) Ctrl+Shift+F10

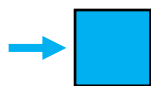
原理：去背底图 (C) = A - B

原图 (A)



max_scale (w)

.....



min_scale ($\frac{w}{2^n}$)

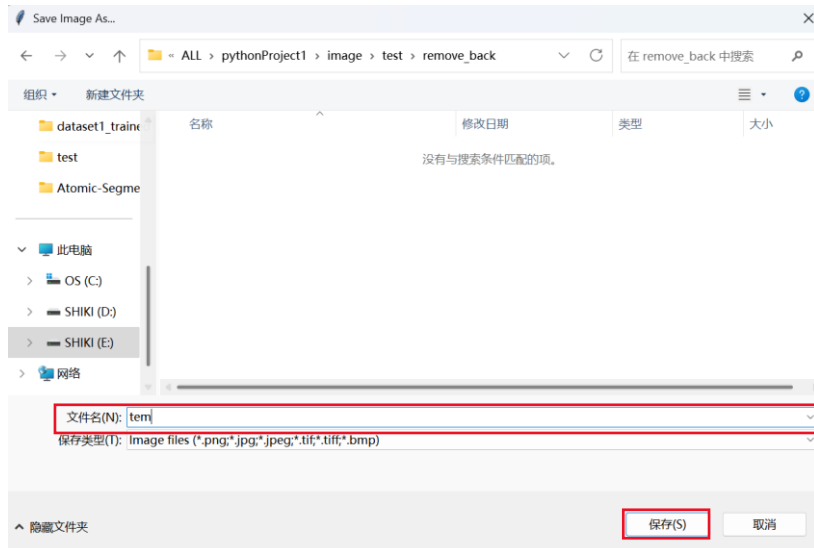
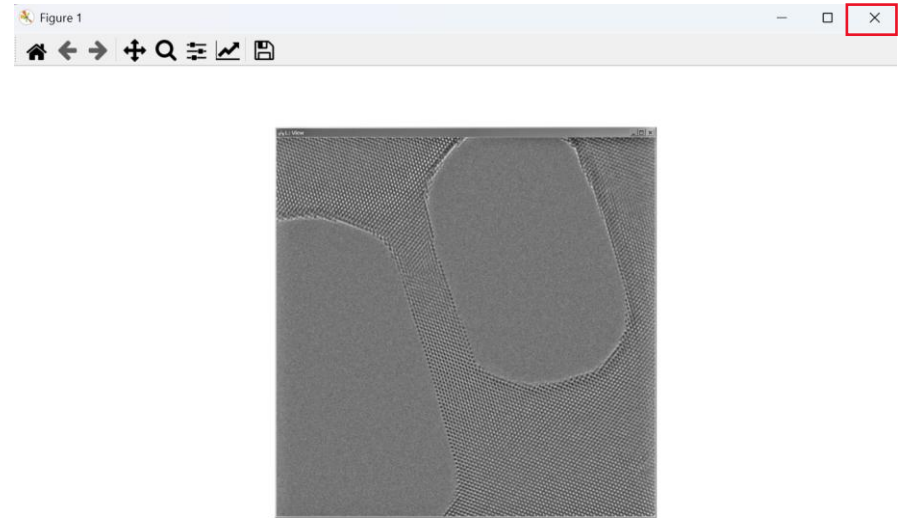
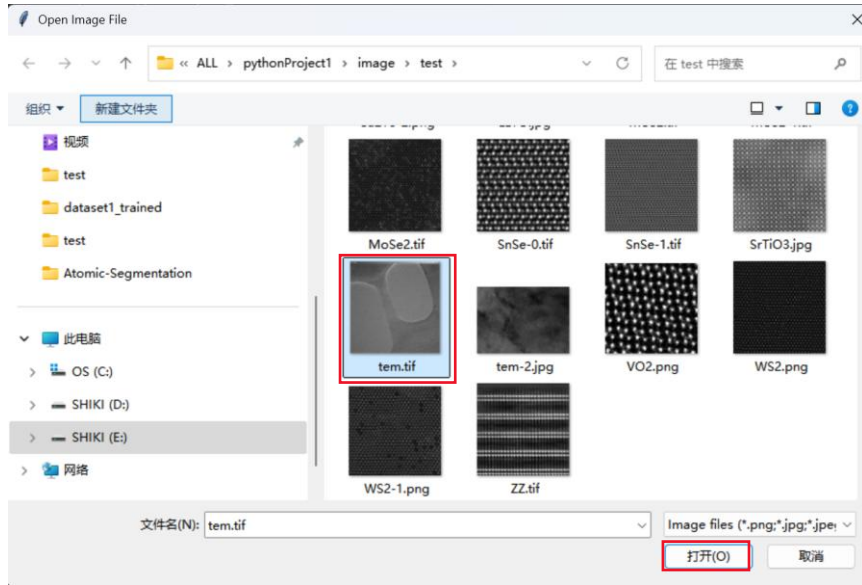
.....



背底 (B)

max_scale (w)

STEM图像去背底



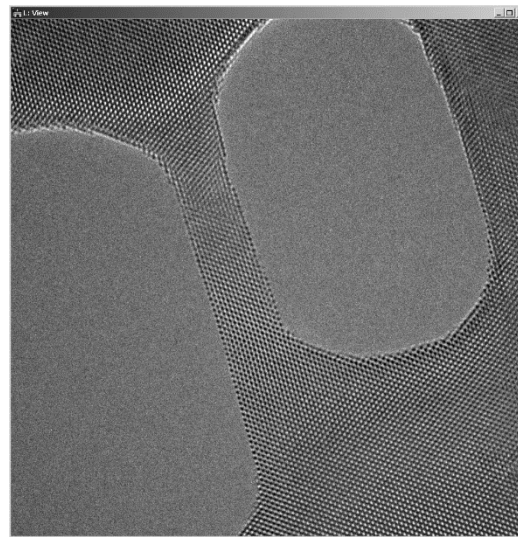
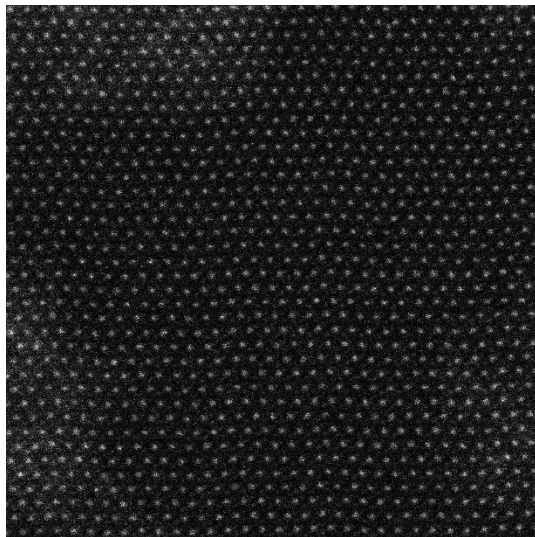
流程:

选择图像—预览输出图像—选择存储路径

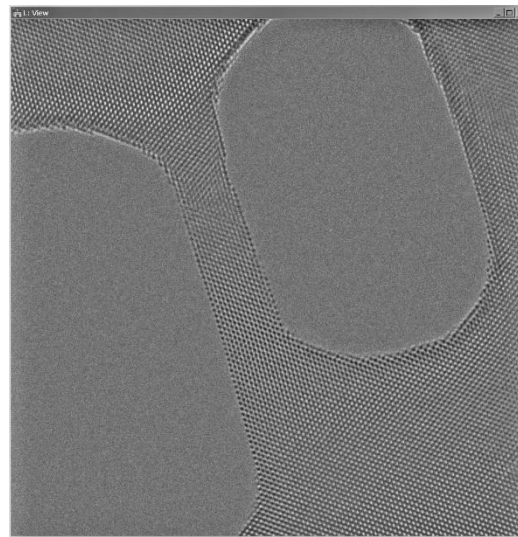
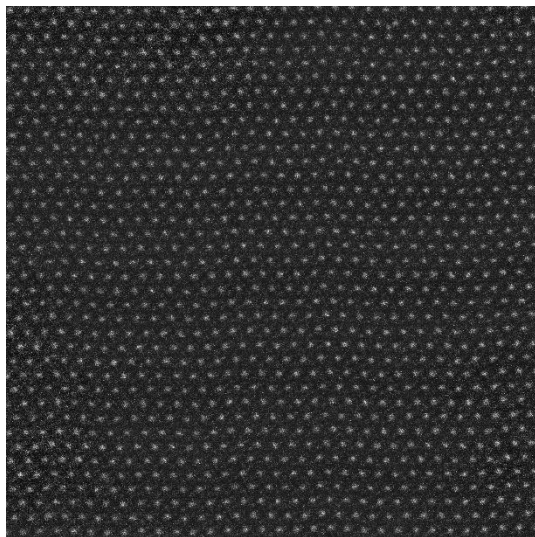
STEM图像去背底

效果展示:

原图



处理后



目录

1、STEM图像处理

1.1 去背底

1.2 降噪

1.3 应变分析

1.4 元素分析

STEM图像降噪

remove_denoise.py ×

```
1 from image.pkg import Backgroundremover, load_img, Prediction, Average_img
2 from model.unet.unet import Unet
3 from model.unet.unet3 import Unet as Unet3
4
5 # load Image
6 image = load_img()
7
8 # remove background:
9 # Define the scaling parameters and remove the background from the image
10 remover = Backgroundremover(image, max_scale=2048, min_scale=16)
11 image = remover.remove()
12 remover.visualize() # 可视化去背景后的图像
13
14 periodicity = 2
15 stride = 6/8
16 batch_size = 8
17 # periodicity: number of boxes in one row or column; stride: stride of the sliding window;
18 # model: Unet(n_classes=1); batch_size: batch size for prediction
19 denoiser = Prediction(image, periodicity, stride, Unet(n_classes=1), batch_size)
20 ori_image, denoised_image = denoiser.denoise()
21 denoiser.visualize(denoised_image) # 可视化去噪后的图像
22 # denoiser.save(denoised_image)
23
24 Average_img(denoised_image, ori_image, windows_size = 768)
```

去背底 (略)

去噪

图像叠加

适用范围: STEM图像 (任意尺寸)

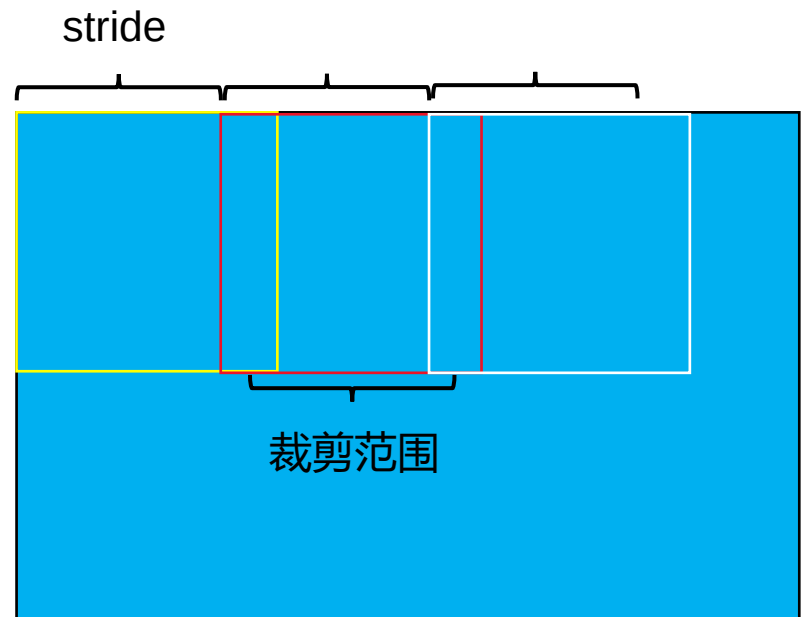
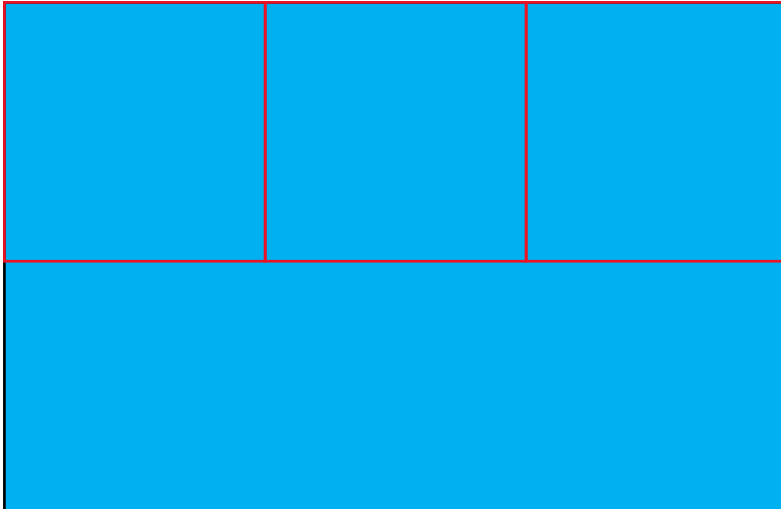
使用方法: 右键—运行

STEM图像降噪

主要参数

```
14 periodicity = 2
15 stride = 6/8
16 batch_size = 8
```

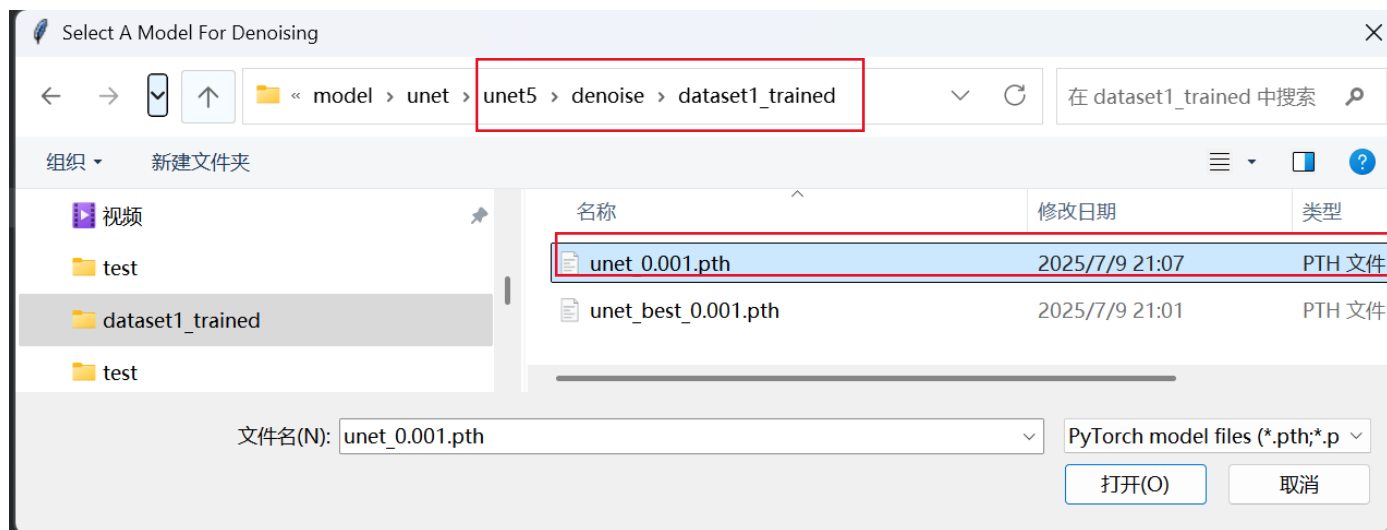
periodicity (eg.3 as below)



STEM图像降噪

流程：

- 1、选择图像—2、预览去背底图像—3、选择模型权重（权重和选择的模型对应）
- 4、预览去噪图像



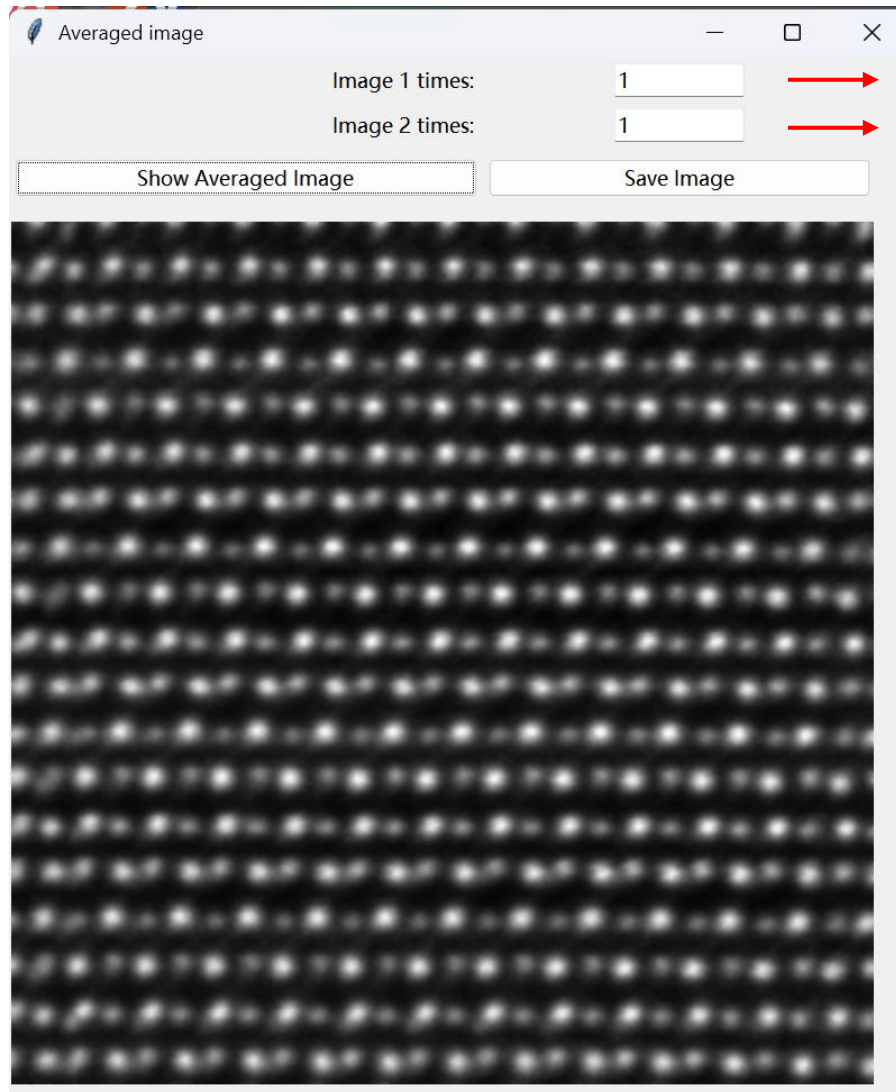
```
19 denoiser = Prediction(image, periodicity, stride, Unet5(n_classes=1), batch_size)
```

Unet3 or Unet5

Unet3更快，一张图片1s以下； Unet5稍慢，约几秒

STEM图像降噪

5、图像叠加



去噪图像叠加次数

原图叠加次数

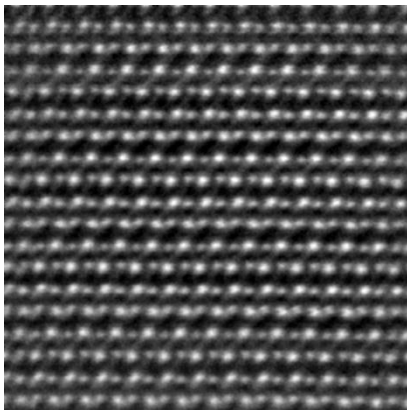
Show Averaged Image
(点击预览)

Save Image
(点击保存)

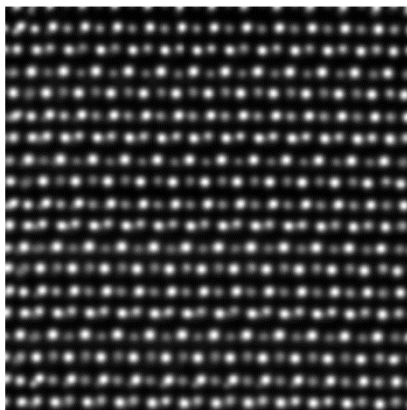
STEM图像降噪

效果展示:

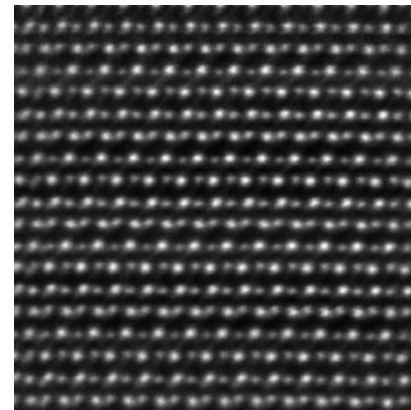
原图 (SnSe)



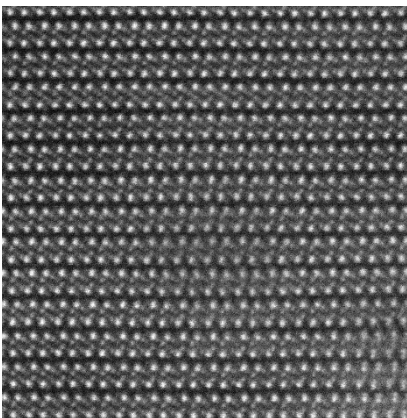
降噪



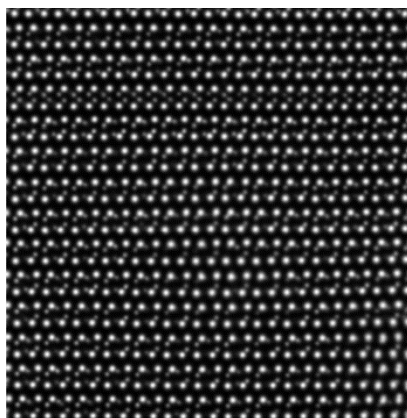
图像1:1叠加



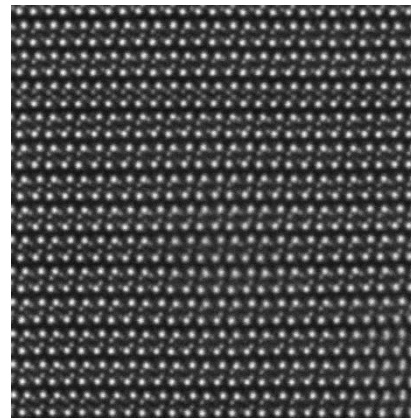
原图 (Cu₂Te)



降噪



图像1:1叠加



目录

1、STEM图像处理

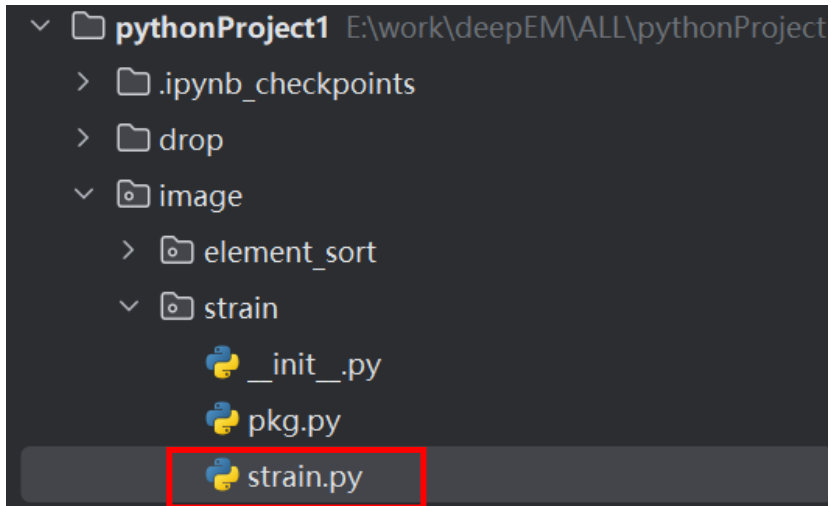
1.1 去背底

1.2 降噪

1.3 应变分析

1.4 元素分析

应变分析



1、选择图片—去背底 (略)

```
5 # Load the original STEM image
6 image = load_img()
7
8 # Step 1: Remove the background from the image
9 # max_scale and min_scale define the scale range for background removal
10 remover = Backgroundremover(image, max_scale=2048, min_scale=32)
11 image = remover.remove()
12 remover.visualize() # Visualize the image after background removal
```

应变分析

2、去噪和位置检测

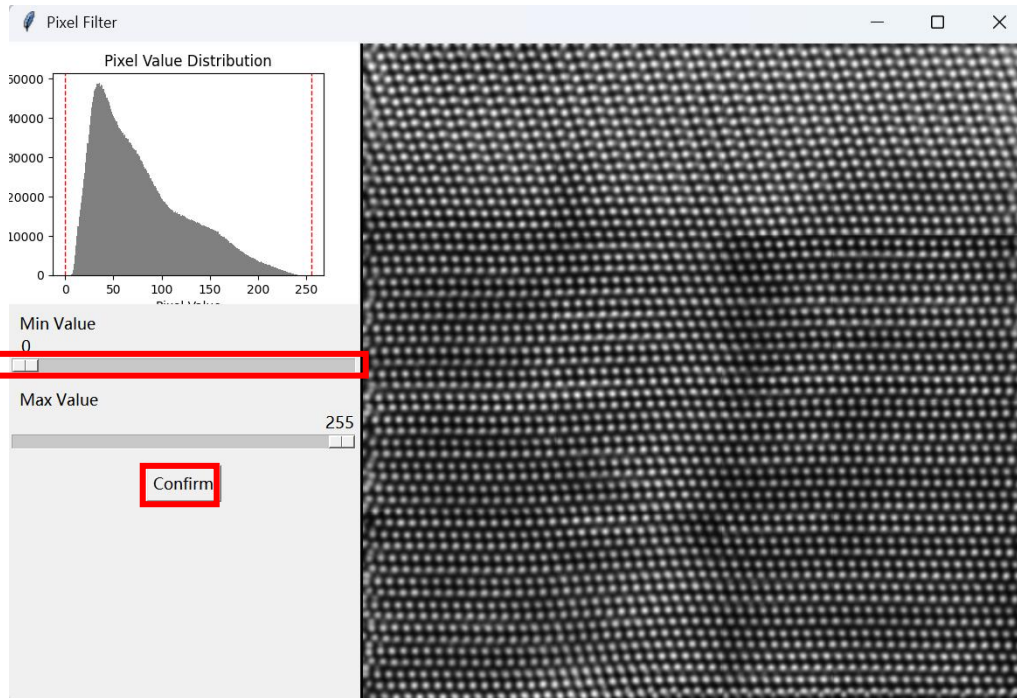
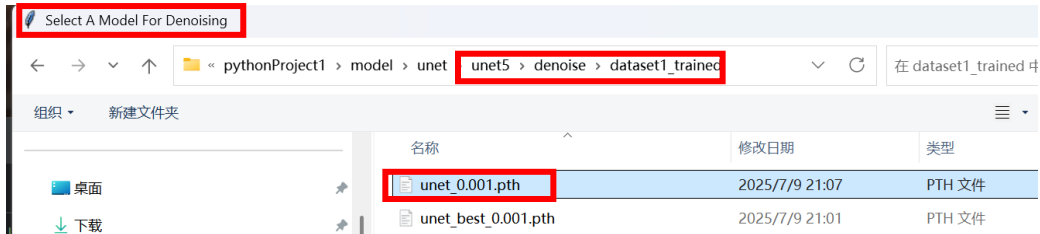
```
# Step 2: Denoise the image and detect atomic positions
# periodicity: number of boxes per row/column (the bigger one) for denoising and segmentation
# stride: step size of sliding window
# batch_size: batch size for the denoising model
# model: model type for denoising ('unet' or 'unet3')
# split_method: method to split denoised regions ('watershed' or 'floodfill')
# min_distance: minimum distance between detected atoms
# filter_ratio: ratio threshold to filter weak detections (usually 0.5-1)
# I: intensity metric to consider ('i_peak' or 'i_mean')
denoised_image, filtered_image, cropped_image, filtered_cropped_image, atoms, atoms_crop = Denoise_Position(
    image,
    periodicity=3,
    stride=6/8,
    batch_size=8,
    model='unet',
    split_method='floodfill',
    min_distance=3,
    filter_ratio=0.8,
    I='i_peak',
    save=False
)
```

图像去噪和分割参数设置

原子识别参数设置

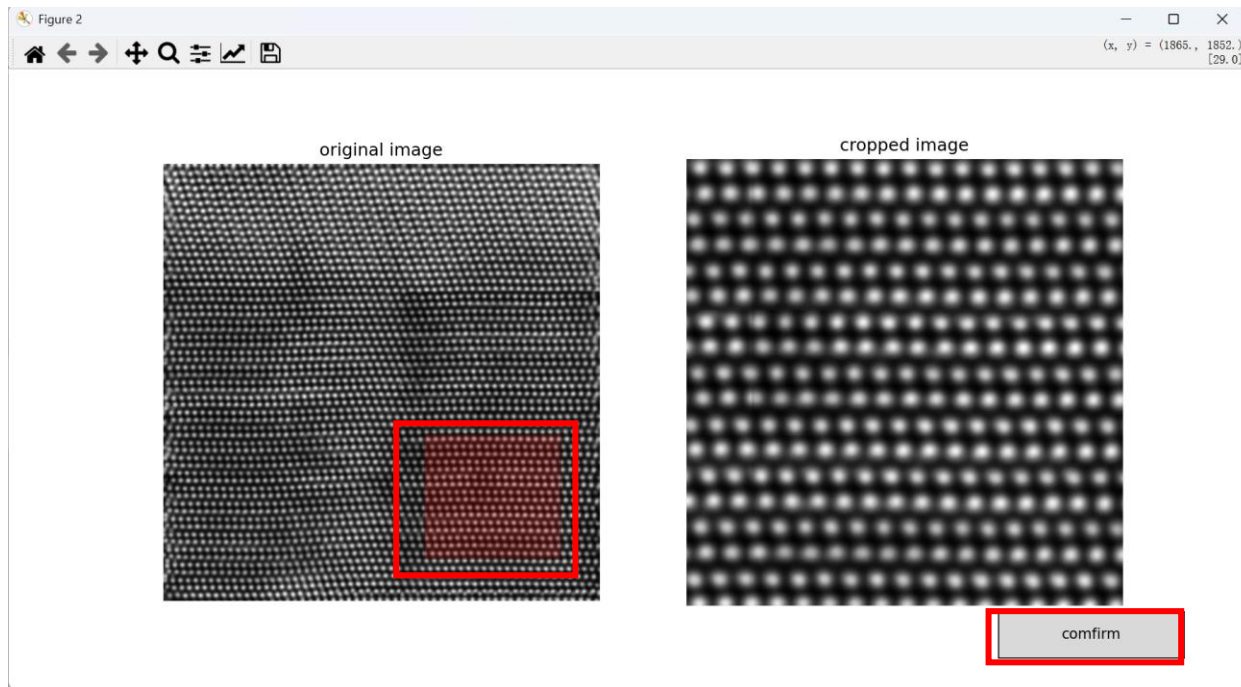
应变分析

2.1、去噪--像素滤除（尽可能滤除非框架原子）



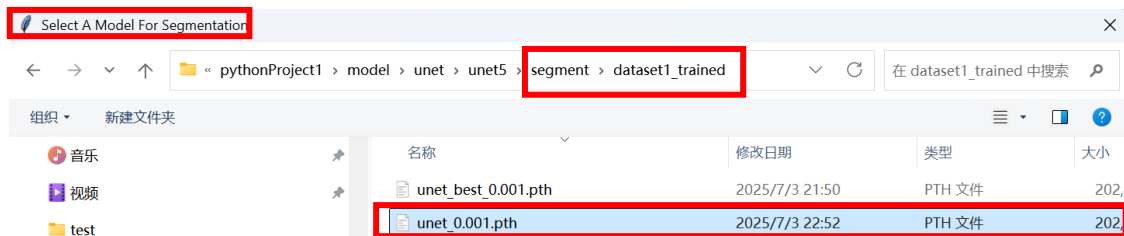
应变分析

2.2、选择标准区域（畸变最小的区域）

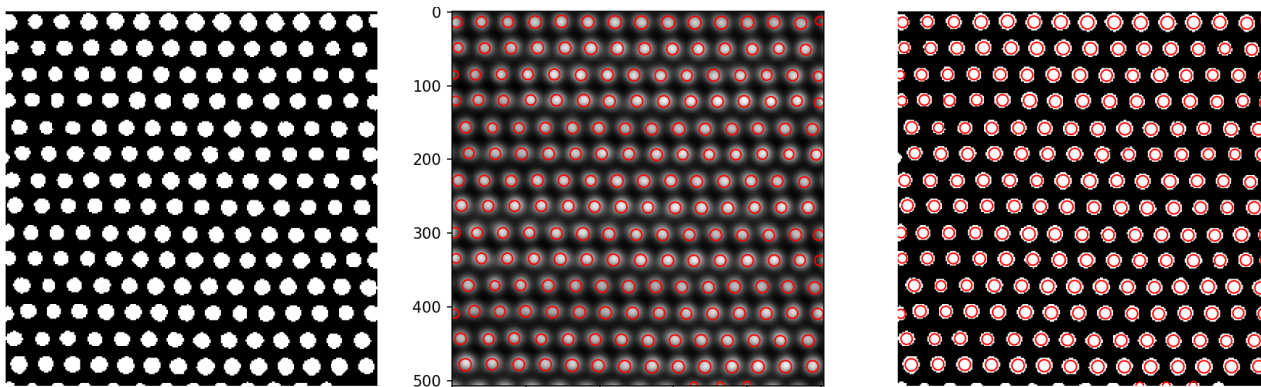


应变分析

2.3、原子识别模型权重



标准区域原子识别结果预览



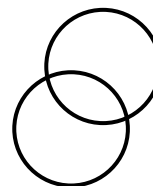
相关输入参数调整:

split_method: 检测算法 ('watershed' or 'floodfill')

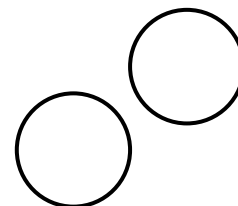
min_distance: 原子间最小间距

filter_ratio: 滤除半径超出限制原子 $\bar{r} \cdot \left(1 - \frac{k}{2}\right) < r < \bar{r} \cdot (1 + k)$

watershed

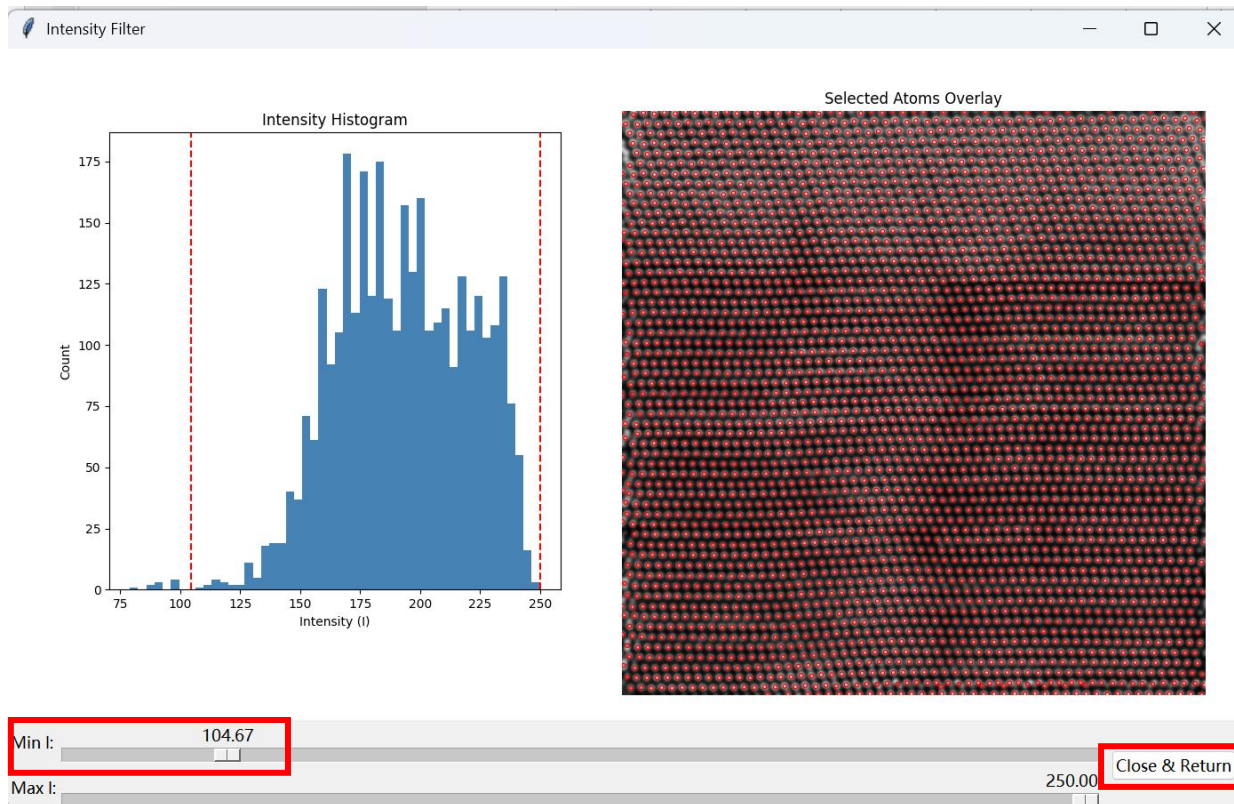


floodfill



应变分析

2.4、选择像素在一定范围的原子（尽可能滤除非框架原子）



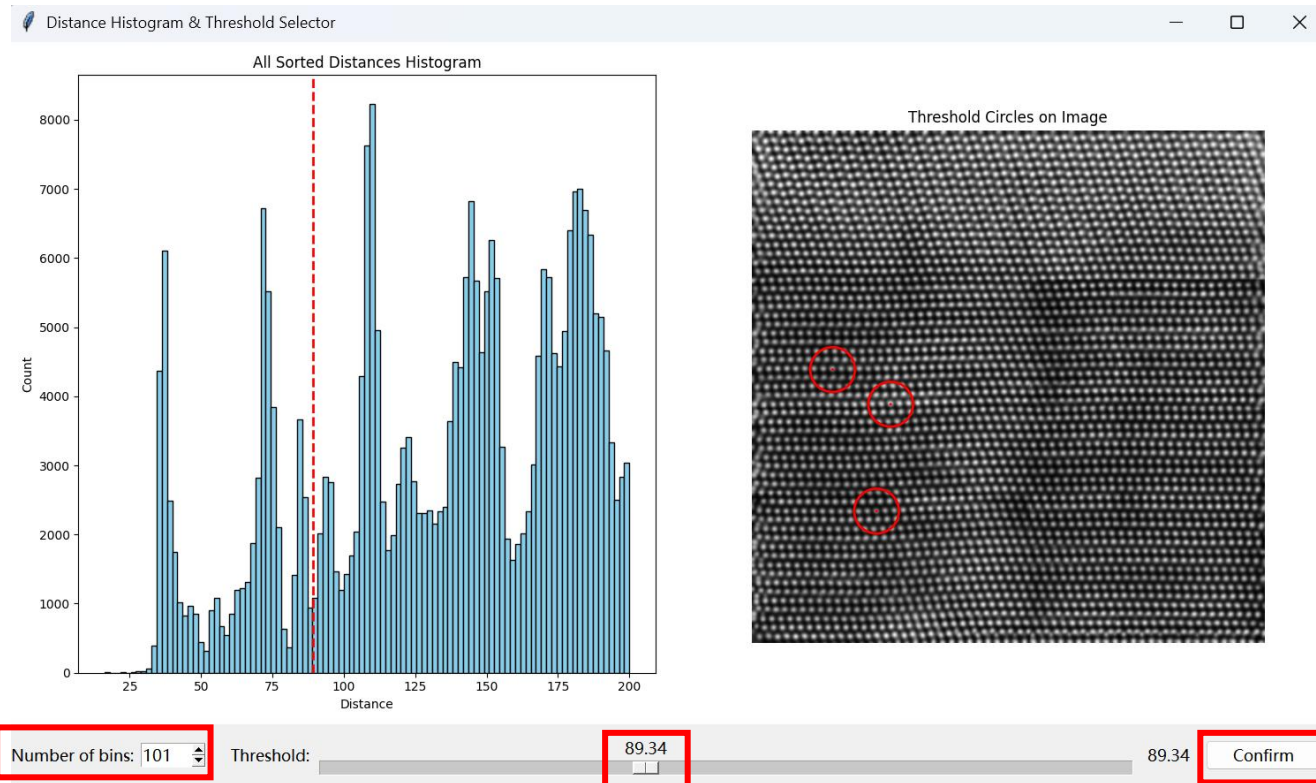
相关输入参数调整：

I：根据峰值像素还是平均像素进行滤除（i_peak or i_mean）

应变分析

3、选择近邻原子范围（在一组基矢方向都要大于一个周期）

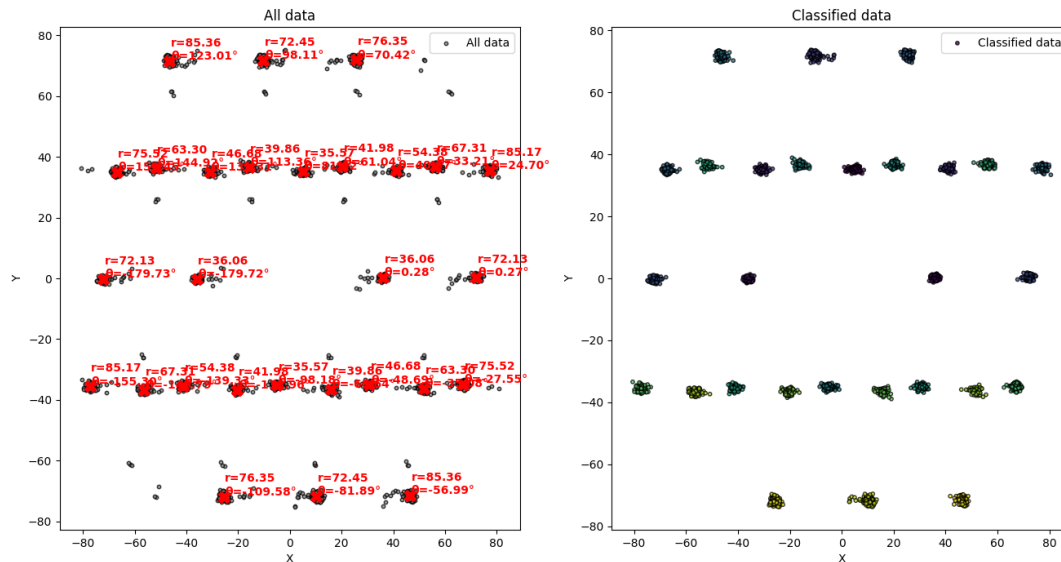
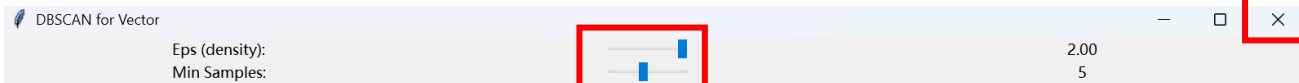
```
36 # Step 3: Calculate neighboring atoms for each detected atom within radius r
37 # neighbor_dis: distances to neighbors
38 # neighbor_pos: neighbor positions
39 # cutoff: distance cutoff threshold
40 r = 200 # Neighbor search radius in pixels (or unit matching image scale)
41 neighbor_dis, neighbor_pos, cutoff = Calculate_neighbor(atoms.to_numpy(), r, denoised_image)
42 neighbor_dis_crop, neighbor_pos_crop, _ = Calculate_neighbor(atoms_crop.to_numpy(), r, cropped_image, if_crop=True)
```



应变分析

4、对基矢进行聚类（标准区域）

```
44  # Step 4: Classify standard lattice vectors and label atoms in the cropped region
45  # vector_centers: centers of standard lattice vectors
46  # atoms_labels_cropped: labels assigned to atoms in cropped region based on vector classification
47  vector_centers, atoms_labels_cropped = Vector_centers(
48      atoms_crop.to_numpy(), neighbor_dis_crop, neighbor_pos_crop, cutoff
49  )
```



Refine

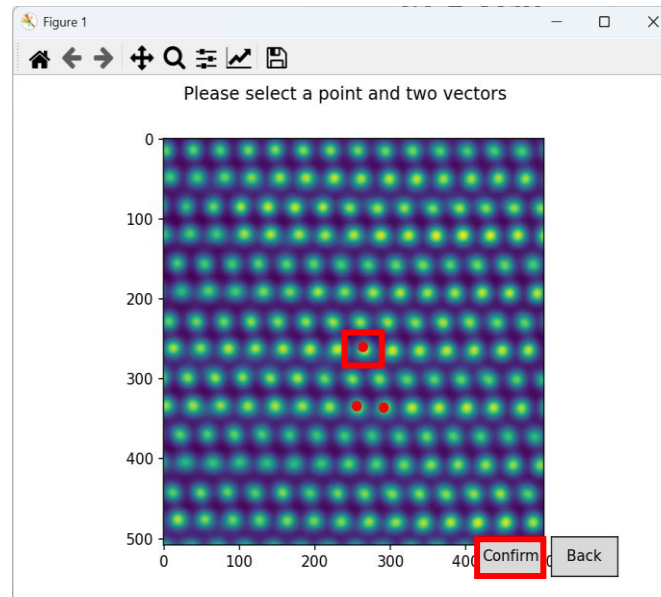
Save Image

应变分析

5-6、根据基矢对原子进行分类，并选择其中一类进行应变分析（同一元素不同空间位置）

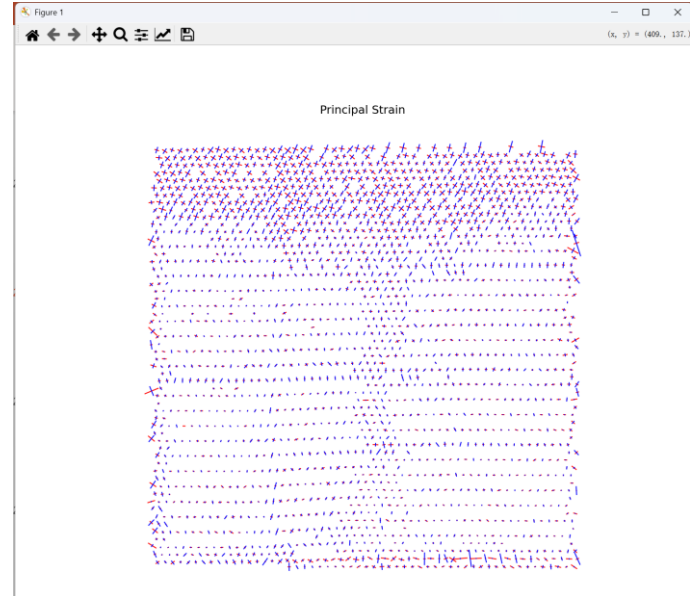
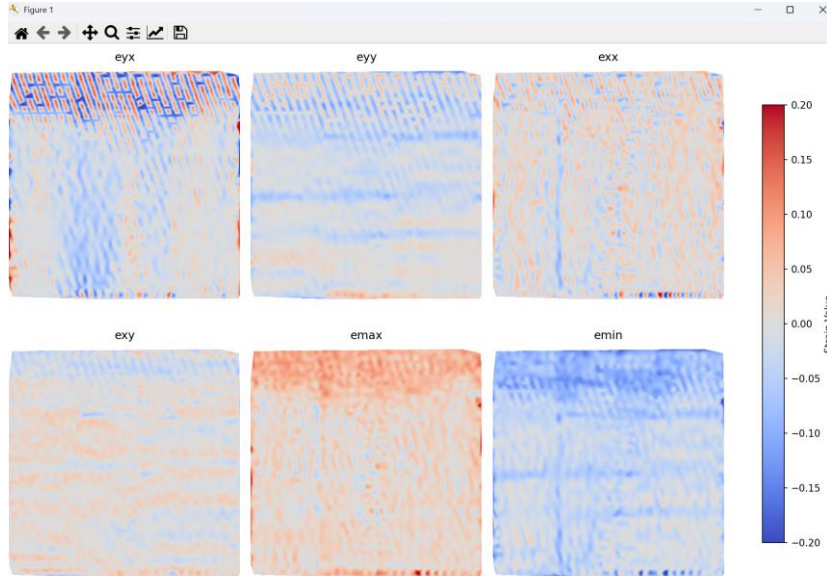
7、选择一组基矢（第一个点为O，第二三个为A,B，组成OA,OB基矢）

```
51 # Step 5: Extract lattice vectors for all atoms in the original image and classify them
52 # vectors_list: list of lattice vectors for each atom
53 # vector_labels_list: labels for each lattice vector
54 # atoms_labels: atom labels based on lattice vector classification
55 vectors_list, vector_labels_list, atoms_labels = classify_vectors(
56     vector_centers, atoms.to_numpy(), neighbor_dis,
57     neighbor_pos, cutoff, dist_multi=1)
58
59 # Step 6: Interactive atom selector GUI for the user to select specific lattice vector classes
60 # selected_vectors_list: lattice vectors selected by user
61 # selected_vector_labels_list: labels of selected vectors
62 # selected_atoms: atom positions corresponding to the selected vectors
63 atomselector = AtomSelector(vectors_list, vector_labels_list, atoms.to_numpy(), atoms_labels, denoised_image)
64 selected_vectors_list, selected_vector_labels_list, selected_atoms = atomselector.get_result()
65
66 # Step 7: Define a set of standard lattice vectors for strain calculation
67 standard_a, standard_b, label_a, label_b = standard_vector(cropped_image, vector_centers)
```



应变分析

8、应变分析结果



9、改变color bar最大值重新绘图

```
***** Calculate Strain *****  
  
Enter new color bar max value (or press Enter to exit): 0.15  
  
Enter new color bar max value (or press Enter to exit): 0.2
```

应变分析

补充：输出日志

```
运行 strain (1) x
E:\conda\Anaconda3\envs\video\python.exe E:\work\deepEM\ALL\pythonProject1\image\strain\strain.py
Image has been read: E:/work/deepEM/ALL/pythonProject1/image/test/Cu2Se.tif

***** Start denoising and position detection *****
Model loaded from: E:/work/deepEM/ALL/pythonProject1/model/unet/unet5/denoise/dataset1_trained/unet_0.001.pth
Total denoise cost time: 6.996891975402832 s; Number of patches:16
E:\work\deepEM\ALL\pythonProject1\image\strain\pkg.py:45: UserWarning: Attempting to set identical low and high
ax2.imshow(cropped_img, cmap='gray')
E:\work\deepEM\ALL\pythonProject1\image\strain\pkg.py:45: UserWarning: Attempting to set identical low and high
ax2.imshow(cropped_img, cmap='gray')
Model loaded from: E:/work/deepEM/ALL/pythonProject1/model/unet/unet5/segment/dataset1_trained/unet_0.001.pth
Number of patches: 16
Total segmentation cost time: 0.9736247062683105 s; Number of patches:16
Model loaded from: E:/work/deepEM/ALL/pythonProject1/model/unet/unet5/segment/dataset1_trained/unet_0.001.pth
Number of patches: 1
Total segmentation cost time: 0.36517810821533203 s; Number of patches:1
检测到3318个原子
检测到205个原子
Filtered atoms in original image: 3308, Filtered atoms in cropped image: 205
***** Denoising and position detection completed *****

***** Atoms Classify *****
Selected cut-off distance: 89.34
The number of vectors: 2
Selected point: [255 334]
Vector 1: [36 1]
Vector 2: [ 8 -75]

***** Calculate Strain *****

Enter new color bar max value (or press Enter to exit): 0.15

Enter new color bar max value (or press Enter to exit): 0.2
```

目录

1、STEM图像处理

1.1 去背底

1.2 降噪

1.3 应变分析

1.4 元素分析

元素分析

选择图片—去背底—降噪—原子识别 (略)

```
element.py x
1 from image.pkg import Backgroundremover, Prediction, Position, Element, load_img
2 from model.unet.unet import Unet as Unet
3 from model.unet.unet3 import Unet as Unet3
4 import numpy as np
5
6 image = load_img()
7
8 #remove background
9 remover = Backgroundremover(image, max_scale=2048, min_scale=32)
10 image = remover.remove()
11 remover.visualize() # 可视化去背景后的图像
12
13 periodicity = 2
14 stride = 6/8
15 batch_size = 8
16 # periodicity: number of boxes in one row or column; stride: stride of the sliding window;
17 # batch_size: batch size for prediction
18 denoiser = Prediction(image, periodicity, stride, Unet3(_classes=1), batch_size)
19 ori_image, image = denoiser.denoise()
20 denoiser.visualize(image) # 可视化去噪后的图像
21
22 segmenter = Prediction(image, periodicity, stride, Unet3(), batch_size)
23 image, output = segmenter.segment()
24 segmenter.visualize(output) # 可视化分割后的图像
25
26 #二值化,检测原子位置
27 output_np = np.array(output.convert("L")).astype(np.uint8)
28 ori_image_np = np.array(ori_image.convert("L")).astype(np.float32)
29 #atoms为一个DataFrame: x, y, r, fit_area, ori_area, i_mean, i_peak
30 #split_method = 'watershed'/'floodfill'
31 position_detector = Position(ori_image_np, output_np, erosion_iter=0,
32                             split_method='floodfill', min_distance=5, filter_ratio=0.8)
33 atoms = position_detector.position_detect()
34 position_detector.visualize(atoms, save_path=None)
```

元素分析

元素分析

```
36 # 检测元素
37 element_detector = Element(ori_image_np, atoms, I='i_peak', start_bins=20, bin_step=20,
38                             min_prominence_ratio=0.25, smooth_sigma=2)
39 atoms = element_detector.element_detect()
40 element_detector.visualize(save_path=None,
41                             colors=['red', 'blue', 'orange', 'green', 'purple', 'brown', 'pink'])
42
```

相关输入参数调整：

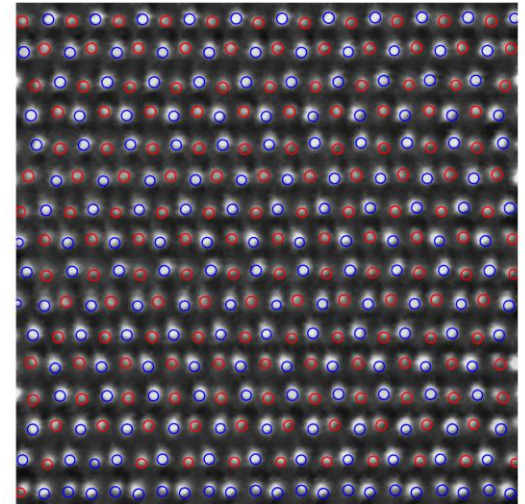
I: 根据峰值像素还是平均像素进行分类 (i_peak or i_mean)

start_bin: 直方图初始bin数

bin_step: bin增加步长

min_prominence_ratio: 滤除较小的峰

smooth_sigma: 高斯拟合参数



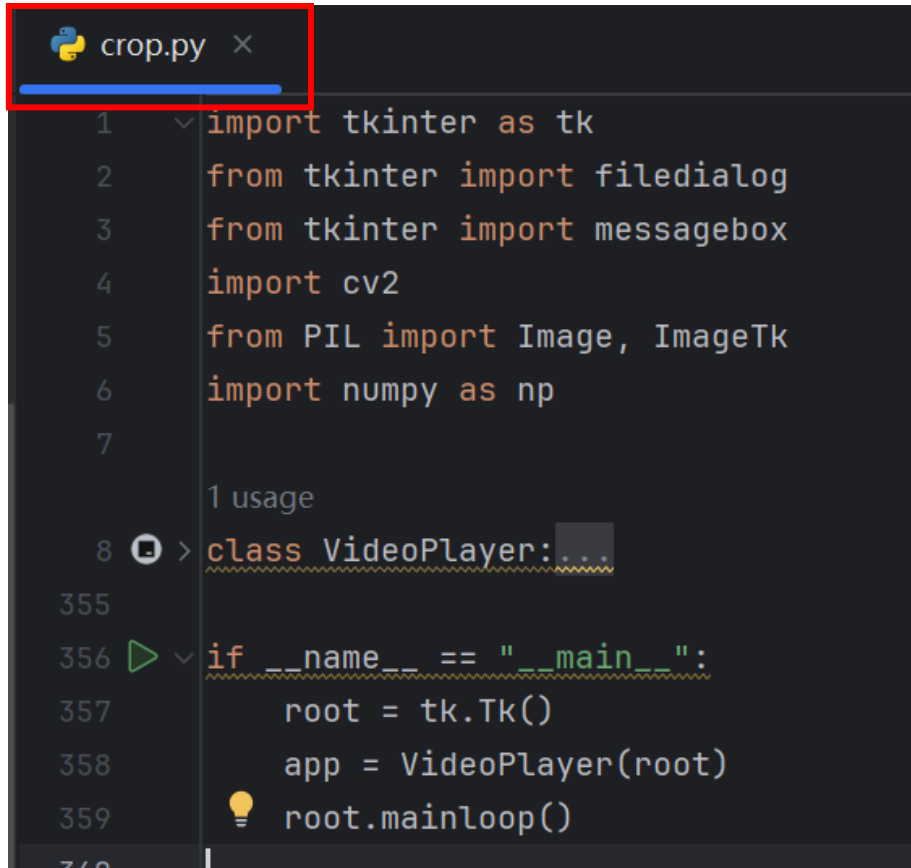
目录

2、STEM视频处理

2.1 裁剪

2.2 抽帧

STEM视频裁剪

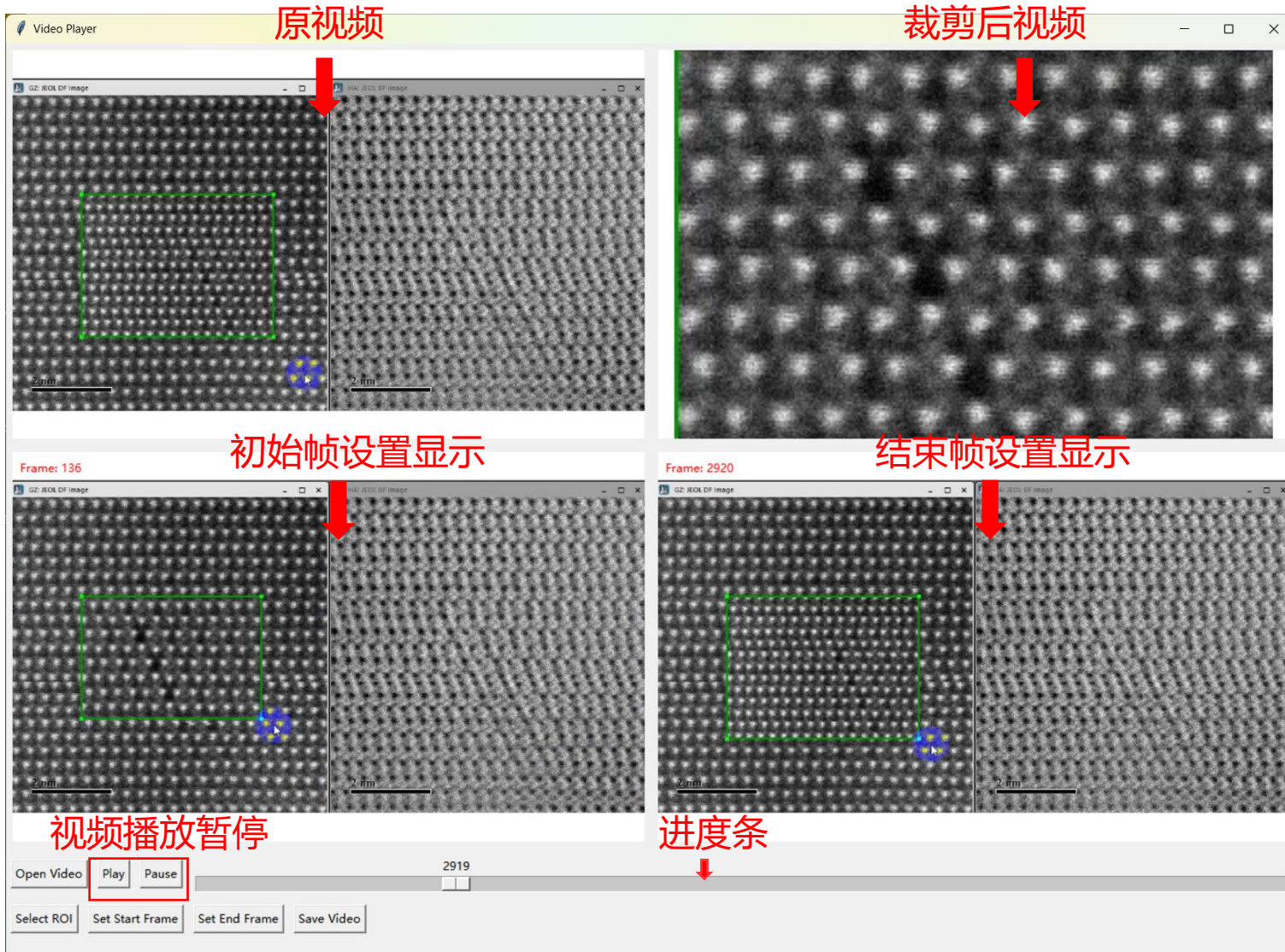


```
1 import tkinter as tk
2 from tkinter import filedialog
3 from tkinter import messagebox
4 import cv2
5 from PIL import Image, ImageTk
6 import numpy as np
7
8 1 usage
9 > class VideoPlayer:...
355
356 > if __name__ == "__main__":
357     root = tk.Tk()
358     app = VideoPlayer(root)
359     root.mainloop()
```

适用范围：STEM视频（任意尺寸）

使用方法：右键—运行

STEM视频裁剪



流程

- 1、Open Video (加载视频)
- 2、Select ROI (选择裁剪区域)
- 3、Set Start Frame (开始帧)
- 4、Set End Frame (结束帧)
- 5、Save Video (输出视频)

目录

2、STEM视频处理

2.1 裁剪

2.2 抽帧

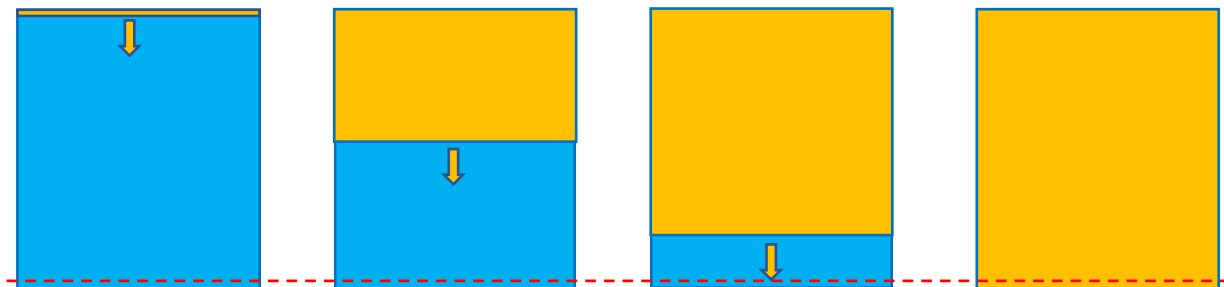
STEM视频抽帧

```
refine.py x
1 > import ...
9
1 usage
10 > def calculate_bottom_difference(frame1, frame2, num_rows=3):...
15
1 usage
16 > def refine_video(input_path, threshold=1, num_rows=3):...
48
1 usage
49 > class VideoApp:...
382
383 > if __name__ == '__main__':...
387
```

适用范围：STEM视频（任意尺寸）

使用方法：右键—运行

原理：

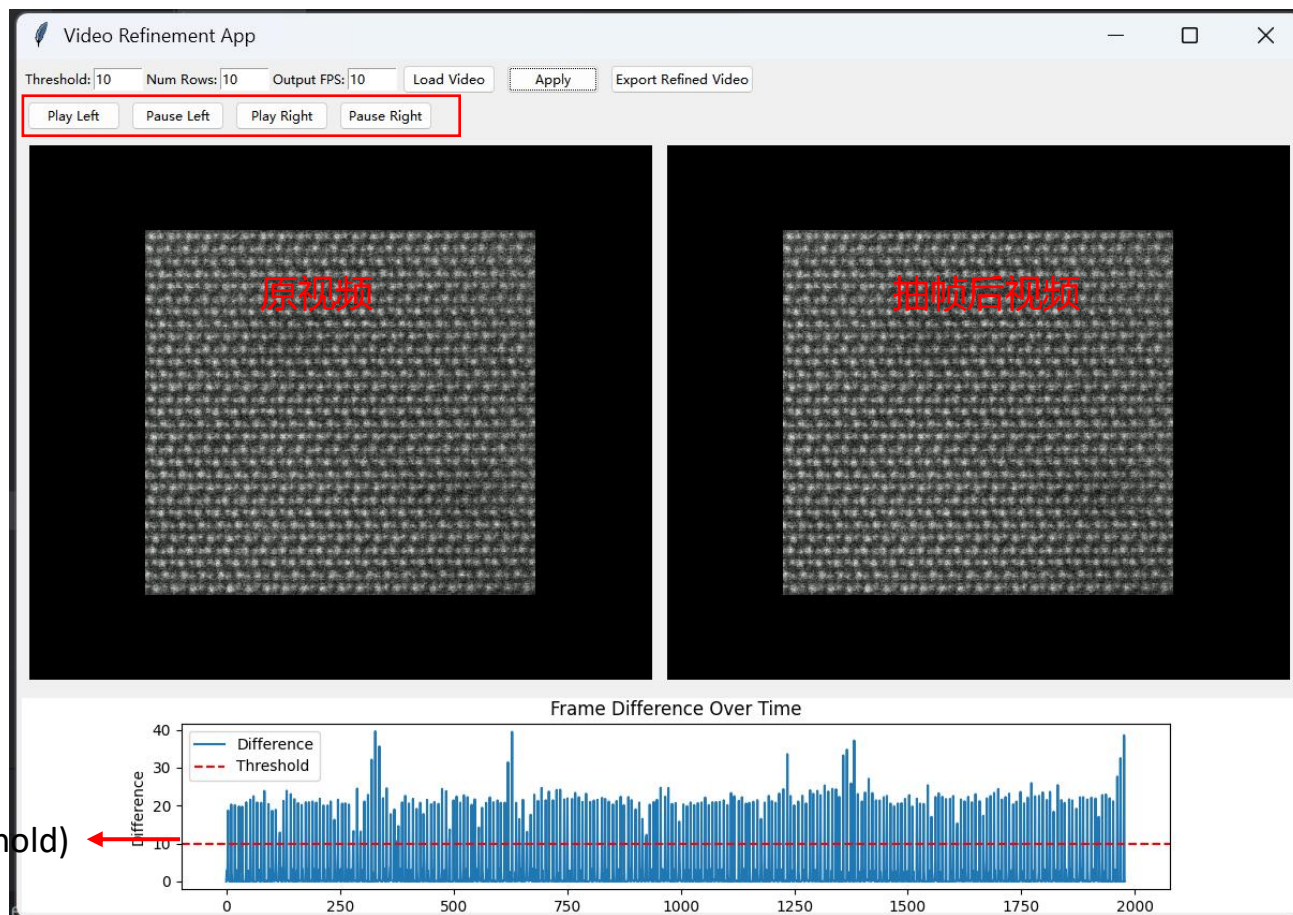


判定条件：

$$A_t - A_{t-1} > \Delta$$

STEM视频抽帧

视频播放



$$A_t - A_{t-1} (\text{对于最后Num Rows行})$$

流程

- 1、Load Video
(加载视频)
- 2、调整Threshold和
Num Rows
- 3、Apply
- 4、调整Output FPS
(输出帧率)
- 5、Export
(输出视频)